

Ensino de Padrões de Projeto utilizando o modelo *Four-Component Instructional Design*: um relato de experiência

João Batista Santos
Universidade Federal do Maranhão
São Luís, Brasil
jbss.neto@discente.ufma.br

Rodrigo Nascimento
Universidade Federal do Maranhão
São Luís, Brasil
rodrigo.nascimento@discente.ufma.br

Lucas Reis
Universidade Federal do Maranhão
São Luís, Brasil
lucas.reis@discente.ufma.br

Luis Rivero
Universidade Federal do Maranhão
São Luís, Brasil
luis.rivero@ufma.br

Davi Viana
Universidade Federal do Maranhão
São Luís, Brasil
davi.viana@ufma.br

Resumo

Os padrões de projeto oferecem soluções bem estabelecidas para problemas comuns em software. Os princípios de projeto visam tornar o software mais compreensível, flexível e de fácil manutenção. Os estudantes frequentemente têm dificuldade em compreender a relação entre padrões/princípios abstratos e aplicações do mundo real, especialmente se ainda não dominaram os fundamentos da programação orientada a objetos. Além disso, há uma falta de exemplos práticos contextualizados ou atividades que ilustrem claramente os problemas reais que esses padrões ou princípios se destinam a resolver. Este artigo apresenta um relato de experiência sobre a implementação e avaliação de uma abordagem pedagógica para o ensino de padrões e princípios de projeto baseada no modelo *Four-Component Instructional Design* (4C/ID). A intervenção foi conduzida por meio de uma série de atividades incrementais incorporadas a uma disciplina de Engenharia de Software, combinando estrategicamente fundamentos teóricos com atividades práticas para aprimorar a assimilação conceitual. Uma avaliação foi realizada por meio da análise do desempenho na disciplina e do feedback dos estudantes, revelando o impacto positivo da prática estruturada nos resultados de aprendizagem. Os achados sugerem que atividades práticas bem planejadas podem aprimorar significativamente a capacidade dos estudantes de compreender e aplicar padrões de projeto de forma eficaz, oferecendo insights valiosos para futuros refinamentos dessa abordagem instrucional.

Palavras-chave

Padrões de projeto, Prática de ensino, Programação Orientada a Objetos, *Four-Component Instructional Design*

1 Introdução

Aprender a programar não é uma atividade trivial e, por consequência, ensinar a programar mantendo a qualidade do software é uma tarefa desafiadora que requer do docente uma composição de estratégias pedagógicas apoiada na resolução de problemas [5]. Esses desafios também são encontrados na criação de software reutilizável e flexível utilizando programação orientada a objetos [9]. Uma solução típica para esse problema são os princípios e padrões de projeto, que auxiliam na criação de software de qualidade.

Um padrão de projeto (do inglês *design pattern*) descreve um problema e como solucioná-lo, considerando objetos e interfaces

[24]. Gamma et al. [9] definem um conjunto de 23 padrões de projeto, considerados peças centrais na resolução de problemas de projeto no desenvolvimento de software [2], utilizando melhores práticas e os princípios de qualidade do projeto orientado a objetos [20]. Por sua vez, os princípios são diretrizes que visam melhorar a qualidade do código em sistemas orientados a objetos, tornando o software mais flexível e escalável, viabilizando a manutenção simplificada e proporcionando um código mais legível e compreensível [16]. Os princípios SOLID (do inglês, *Single responsibility, Open-closed, Liskov substitution, Interface segregation e Dependency inversion*) são exemplos utilizados na indústria de software [18]. Embora a empregabilidade de padrões de projeto auxilie na criação de software reutilizável, flexível e de fácil manutenção, o ensino e a aprendizagem desses padrões são tarefas difíceis [1].

Um dos desafios no ensino de padrões de projeto é a complexidade dos conceitos envolvidos, que exige dos alunos um alto grau de abstração e conhecimento prévio em engenharia de software e programação orientada a objetos. Alunos enfrentam dificuldades para compreender e aplicar esses conceitos de forma eficaz sem uma base sólida em programação [6]. Outro aspecto que contribui para essa dificuldade é a falta de experiência prática, pois muitos estudantes têm uma compreensão teórica dos padrões, mas relutam em aplicá-los nos contextos reais do dia a dia do desenvolvimento de software [4].

O ensino de aspectos que promovam uma melhor qualidade de software, incluindo princípios e padrões de projeto, tem sido estudado na literatura por representar um desafio recorrente devido à necessidade de equilibrar conceitos teóricos e aplicação prática [20]. Entre as diferentes abordagens pedagógicas exploradas para facilitar esse processo, destaca-se o modelo *Four-Component Instructional Design* (4C/ID), que estimula uma aprendizagem teórico-prática [24].

Diante desse cenário, este trabalho apresenta um relato de experiência no ensino de padrões de projeto e princípios SOLID, com foco na aplicação prática e no desenvolvimento das habilidades dos alunos, estruturando o aprendizado em etapas que favoreçam a fixação dos conceitos e sua aplicação no desenvolvimento de software. Este artigo contribui para a educação em Engenharia de Software ao: (i) propor uma estrutura incremental baseada no modelo 4C/ID para o ensino de padrões de projeto e princípios SOLID; (ii) relatar sua aplicação em duas turmas de graduação, incluindo ajustes realizados entre as aplicações; (iii) analisar a percepção dos estudantes

sobre a abordagem por meio de dados quantitativos e qualitativos; e (iv) disponibilizar os artefatos educacionais para apoiar a replicação e adaptação por outros docentes.

O restante deste artigo está organizado da seguinte forma: a Seção 2 explora os trabalhos e conceitos relacionados; a Seção 3 apresenta o método de pesquisa; a Seção 4 contém os resultados alcançados; a Seção 5 discute os resultados obtidos. E, por fim, a Seção 6 apresenta conclusões sobre este trabalho e os trabalhos futuros.

2 Fundamentação e Trabalhos Relacionados

Esta seção apresenta os conceitos e trabalhos relacionados que fundamentam este estudo, incluindo padrões de projeto, princípios SOLID e o modelo 4C/ID, além de discutir as abordagens utilizadas no ensino desses conteúdos.

2.1 Padrões de Projeto

Padrões de projeto são soluções reutilizáveis para problemas comuns no desenvolvimento de software. Pode-se dividir esses padrões em três tipos [9]: de criação, estruturais e comportamentais.

Nesse relato, optou-se por alguns padrões com base em sua relevância e aplicabilidade prática no ensino de desenvolvimento de software. Os padrões e as justificativas de seleção foram: *Singleton*, pois este padrão é fundamental para garantir que uma classe tenha apenas uma instância e é amplamente utilizado em contextos onde um único ponto de controle é necessário [8]; o *Abstract Factory*, este padrão é importante para criar famílias de objetos relacionados sem depender de suas classes concretas [9]; o *Adapter* foi escolhido por sua capacidade de permitir que interfaces incompatíveis trabalhem juntas [8]; o *Composite* é um padrão importante para a composição de objetos em estruturas de árvore para representar hierarquias parte-todo [8]; o *Observer* para implementar notificações automáticas de mudanças de estado [8]; o *Strategy* oferece flexibilidade em encapsular algoritmos e permitir sua intercambialidade, promovendo a reutilização e manutenção do código [8].

Os padrões selecionados neste trabalho foram escolhidos por representarem diferentes categorias de padrões de projeto descritas por Gamma et al. [9], abrangendo padrões criacionais, estruturais e comportamentais. Além disso, foram priorizados padrões amplamente utilizados no ensino e na prática do desenvolvimento orientado a objetos, devido à sua capacidade de exemplificar problemas recorrentes de projeto e soluções reutilizáveis em diferentes contextos de software [1, 10].

2.2 Princípios SOLID

Os princípios SOLID são um conjunto de cinco diretrizes cujo objetivo é a qualidade do código. Esses princípios ajudam a criar software mais flexível, escalável e de fácil manutenção [3]. Neste trabalho, foram abordados os princípios de Responsabilidade Única, Aberto-Fechado e Inversão de Dependência. Sua escolha baseou-se na sua importância para a construção de software flexível e de fácil manutenção.

O Princípio da Responsabilidade Única (SRP) foi selecionado por promover a coesão, garantindo que cada classe tenha uma única responsabilidade [17]. O Princípio Aberto-Fechado (OCP) permite que o software seja entendido sem necessidade de modificar o código

existente, promovendo a reutilização e a robustez do software [15]. E o Princípio da Inversão de Dependência (DIP) foi escolhido por seu papel fundamental no desacoplamento de módulos de alto e baixo nível, aumentando a flexibilidade e a capacidade de adaptação do sistema a mudanças futuras [17].

2.3 Modelo *Four-Component Instructional Design* (4C/ID)

Esse modelo é uma abordagem de *design* instrucional, desenvolvida para lidar com o ensino de tarefas complexas. Ele é baseado em quatro componentes principais: (1) Tarefas de Aprendizagem; (2) Informação de Suporte; (3) Informação Procedural e (4) Prática de Parte-Tarefa. As "Tarefas de Aprendizagem" representam o componente central do modelo e correspondem a atividades planejadas para simular as situações reais, organizadas em níveis progressivos de dificuldade. A "Informação de Suporte" envolve todo o conhecimento teórico necessário para que os alunos possam executar as Tarefas de Aprendizagem, como exemplos e materiais de estudo. Por sua vez, a "Informação Procedural" refere-se ao passo a passo que auxilia os alunos na execução de atividades. Por fim, a "Prática de Parte-Tarefa" consiste na repetição e automatização de componentes específicos das Tarefas de Aprendizagem [24]. Essa abordagem busca reduzir a carga cognitiva dos alunos e favorecer um aprendizado progressivo e integrado [25].

Embora este trabalho adote o modelo 4C/ID, é relevante contrastá-lo com outras abordagens, como o *Problem-Based Learning* (PBL), cuja comparação detalhada é apresentada na Seção 5.1. A escolha pelo 4C/ID neste estudo reflete a necessidade de orientação mais estruturada no ensino de padrões de projeto e dos princípios SOLID, conforme reforçado por experiências anteriores [24].

2.4 Ensino de Padrões de Projeto

O ensino de padrões de projeto é um desafio nas instituições de ensino superior, devido à alta complexidade desses conceitos e à necessidade de maturidade dos alunos em Engenharia de Software e Programação orientada a objetos. Estudos apontam para a importância de estratégias pedagógicas inovadoras para promover a compreensão e a aplicação eficaz dos padrões de projeto [20, 22].

Uma das estratégias é a utilização de projetos práticos, como o desenvolvimento de jogos. Essa abordagem, além de motivar os alunos, também fornece um contexto rico e interdisciplinar no qual os conceitos de padrões de projeto podem ser aplicados de maneira concreta [6]. Além disso, o uso de ferramentas educacionais tem se mostrado relevante para tornar o ensino de padrões de projeto mais dinâmico e acessível. Um exemplo disso é o LEPATTERN, uma ferramenta desenvolvida para auxiliar no aprendizado e na aplicação desses padrões, permitindo que os alunos implementem estruturas de *design* de forma guiada e automatizada [4].

Outra estratégia destacada é a aplicação do modelo 4C/ID, que visa reduzir a carga cognitiva dos alunos ao estruturar o material instrucional em componentes integrados. Esse modelo apresenta indícios de sucesso ao aumentar o engajamento e a compreensão dos alunos sobre padrões de projeto [24]. Silva et al. [20] aplicaram o modelo 4C/ID em um experimento sobre manutenção de

código com e sem o padrão *Strategy*, encontrando resultado contraintuitivo (redução da manutenibilidade). Reis et al. [19] usaram o mesmo modelo para ensinar práticas *DevOps* por meio de cenários progressivos, com ganhos relatados em colaboração e domínio de ferramentas.

Apesar das experiências positivas, é importante reconhecer que o ensino de padrões de projeto deve ser adaptado às necessidades e ao nível de maturidade dos alunos. Alguns estudos sugerem que a introdução desses conceitos deve ser gradual e integrada ao longo do currículo, permitindo que os alunos desenvolvam uma compreensão profunda e prática ao longo do tempo [13]. A estratégia baseada no desenvolvimento de jogos também corrobora essa abordagem gradual, permitindo que os alunos se familiarizem com os padrões de forma progressiva e aplicada [21]. Portanto, a literatura sugere que a escolha das práticas para o ensino de padrões de projeto deve considerar tanto os aspectos motivacionais quanto a eficácia das estratégias aplicadas, permitindo que os alunos não apenas compreendam os conceitos, mas também consigam aplicá-los em seus futuros projetos.

Destaca-se que, diferentemente dos trabalhos da literatura, esse relato de experiência apresenta uma aplicação integrada dos quatro componentes do modelo 4C/ID ao ensino de padrões de projeto, direcionada a duas turmas de engenharia de software, com nenhuma ou pouca experiência, em um curto período de tempo. Essa combinação foi testada para reduzir o peso do ensino e aumentar a aplicação prática dos conceitos, mesmo entre estudantes sem experiência prévia no desenvolvimento de software.

3 Definição da Prática de Ensino e Sua Análise

Neste relato de experiência, foi desenvolvido com base no modelo *Four-Component Instructional Design* (4C/ID). A Figura 1 apresenta o fluxo das atividades realizadas durante a aplicação da prática de ensino. Destacam-se o momento de aplicação dos formulários em cada rodada e o intervalo de seis meses entre elas. A figura também evidencia que a análise dos dados e a escrita deste relato de experiência foram executadas posteriormente à aplicação de ambas as rodadas.

Este relato foi realizado em uma disciplina de Engenharia de Software com alunos do 5º período da Universidade Federal do Maranhão e incluiu aulas teóricas e práticas sobre os princípios SOLID e Padrões de Projeto. Durante as aulas, foram apresentados os conceitos fundamentais desses temas, acompanhados de exemplos de código e diagramas de classes, permitindo que os alunos visualizassem e compreendessem sua aplicação prática. A estrutura do ensino seguiu os quatro componentes do modelo 4C/ID:

- Tarefas de Aprendizagem: foram definidas tarefas principais para os alunos, variando entre duas e três tarefas, conforme a rodada de aplicação:
 - (1) Durante a primeira rodada deste relato, foram definidas duas tarefas. Na primeira tarefa, os alunos escolheram dois padrões de projeto para apresentar, explicando seus conceitos e aplicações por meio de diagramas de classe e exemplos de código. Essa atividade serviu como um primeiro contato prático com os padrões de projeto. A segunda tarefa consistiu em um projeto final, realizado

em grupos de no máximo três integrantes, no qual os alunos receberam duas propostas de projetos previamente desenvolvidos, sem a aplicação de padrões de projeto ou conceitos de código limpo. Eles tinham a possibilidade de escolher apenas um dos projetos e entregar um código refatorado e funcional, bem como o diagrama de classes correspondente, aplicando os princípios SOLID (especificamente, os princípios S, I e D) e pelo menos dois padrões de projeto, como *Singleton*, *Abstract Factory*, *Adapter*, *Composite*, *Observer* ou *Strategy*.

- (2) Na segunda rodada, realizada em outra turma da mesma disciplina, realizaram-se mudanças com base nas sugestões da turma anterior, especialmente sobre a inclusão do desenvolvimento de projetos desde o início. A primeira tarefa consistiu na refatoração de um código legado de um sistema de cadastro simples. Para a segunda tarefa, foi disponibilizada uma lista com seis opções de programas, juntamente com seus requisitos, para os alunos escolherem apenas uma delas para implementar. Na terceira tarefa, realizada em grupos de no máximo três integrantes, os alunos deveriam implementar um sistema de gerenciamento de tarefas utilizando pelo menos dois padrões de projeto entre os mesmos utilizados na primeira rodada, sendo retiradas as exigências de elaboração do diagrama de classes e da aplicação dos princípios SOLID, com o objetivo de permitir maior foco no aprendizado e na implementação dos padrões de projeto.
- Informação de Suporte e Informação Procedural: a informação de suporte abrangeu todo o conhecimento teórico necessário para a realização das tarefas, incluindo aulas expositivas, materiais de apoio e exemplos práticos. Adicionalmente, a informação procedural foi disponibilizada diretamente no contexto das atividades, por meio de guias de implementação, explicações detalhadas durante um acompanhamento no qual os alunos apresentaram seus desenvolvimentos. Eles também podiam esclarecer dúvidas e receber *feedbacks*, além do auxílio extraclasse oferecido pelo grupo de *WhatsApp*, proporcionando suporte contínuo ao aprendizado e à aplicação dos padrões de projeto.
- Prática de Parte-Tarefa: a prática de parte-tarefa esteve presente ao longo do processo, permitindo que os alunos exercitassem partes específicas das tarefas antes da implementação completa. Na primeira rodada, isso ocorreu tanto nas apresentações iniciais, que exigiam a exploração de padrões de projeto isoladamente, quanto na refatoração do código, onde se aplicavam gradualmente os conceitos aprendidos. Na segunda rodada, essa prática foi estruturada em três atividades progressivas: (i) refatoração de um sistema legado de cadastro simples, (ii) implementação de um programa a partir de requisitos previamente definidos e (iii) desenvolvimento de um sistema de gerenciamento de tarefas.

Os projetos de software foram escolhidos com o intuito de minimizar a complexidade, mantendo o foco no aprendizado dos padrões de projeto e dos princípios abordados na disciplina. Dessa forma, buscou-se utilizar propostas acessíveis aos alunos, que ainda permitissem a aplicação prática dos conceitos estudados.

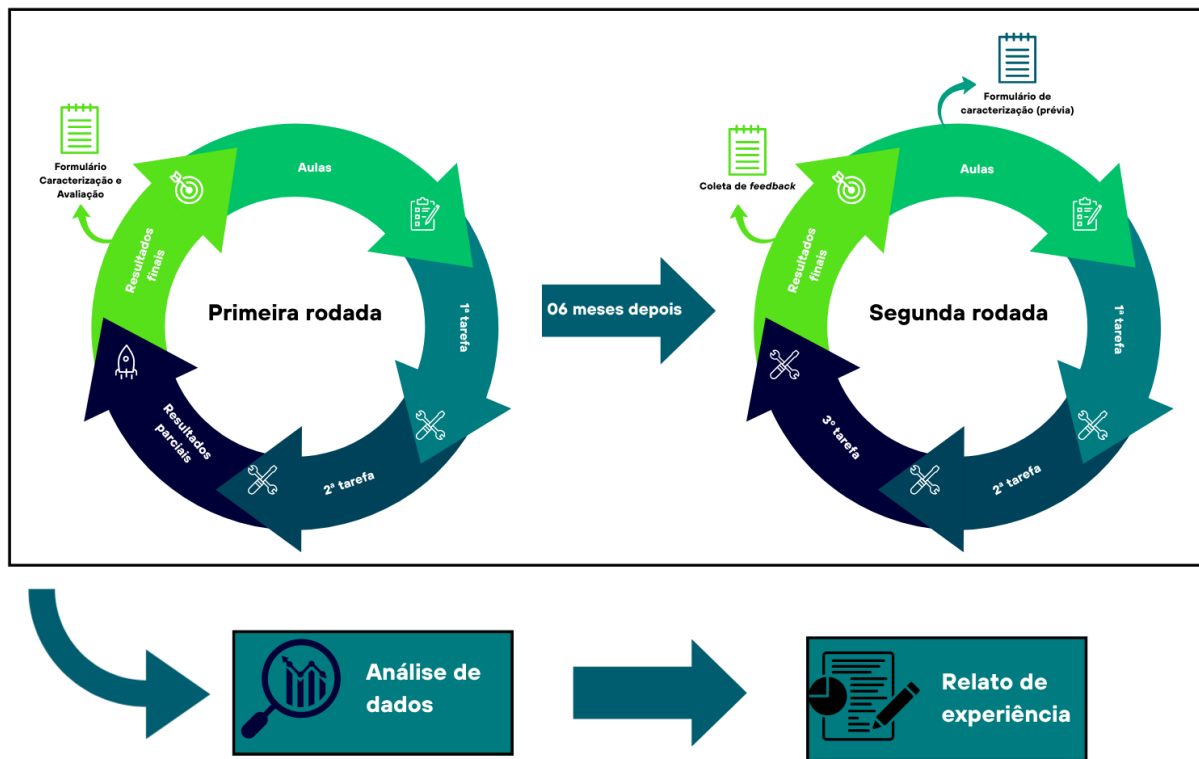


Figura 1: Fluxo das atividades realizadas.

3.1 Coleta de Dados

As questões para a coleta de dados desse relato de experiência foram inspiradas em estudos anteriores sobre o ensino de padrões de projeto [6, 24] e compõem-se de questões fechadas e abertas. As questões fechadas visavam capturar o *feedback* quantitativo dos alunos, enquanto as questões abertas permitiam que os alunos expressassem quais aspectos das atividades contribuíram mais para o aprendizado e fornecessem sugestões para melhorar a prática de ensino.

Na primeira rodada, foi aplicado apenas um formulário de caracterização e avaliação para coletar o contexto dos alunos e *feedback* sobre a prática de ensino. Além de avaliar a percepção dos alunos sobre a didática utilizada e a experiência em aspectos da prática de ensino, o formulário buscou compreender a experiência prévia dos alunos com desenvolvimento de software, programação orientada a objetos e sua familiaridade com padrões de projeto e princípios SOLID.

O formulário da rodada anterior foi dividido em dois formulários para a segunda rodada: um de caracterização dos alunos e outro de avaliação da prática de ensino. Essa alteração foi realizada após a finalização da experiência na primeira rodada, visto que seria importante dividir em dois questionários para obter uma melhor caracterização dos alunos antes da condução da prática de ensino. Nessa segunda rodada, foram adicionadas duas perguntas ao formulário de caracterização com o objetivo de identificar quais linguagens de programação os alunos conheciam e quais ferramentas de desenvolvimento haviam utilizado anteriormente.

A seguir, apresentam-se as definições dos itens de caracterização e de avaliação utilizados nas duas rodadas.

Itens de caracterização:

- **Experiência Prévia com Desenvolvimento de Software:** identifica se o aluno trabalhou na área de desenvolvimento de software, sua familiaridade com o uso de programação orientada a objetos e, adicionalmente, na segunda rodada, quais linguagens e ferramentas o aluno conhece, ajudando a contextualizar suas respostas no restante do formulário.
- **Familiaridade com Padrões de Projeto e Princípios SOLID:** avalia o nível de familiaridade prévia dos alunos com padrões de projeto e os princípios SOLID, proporcionando uma base para entender como o curso impactou o conhecimento existente dos alunos;

Itens de avaliação:

- **Eficiência da Metodologia:** avalia se a abordagem de ensino adotada foi eficaz no aprendizado dos padrões de projeto;
- **Suficiência das Explicações:** mede se as explicações teóricas e os exemplos dados foram adequados para a compreensão dos conceitos;
- **Complexidade das Tarefas:** avalia a percepção dos alunos sobre a dificuldade da tarefa prática, que envolveu a refatoração de código. Devido à inclusão de mais uma tarefa na segunda rodada, foi incluída mais uma questão no formulário sobre dificuldades específicas com a segunda tarefa;

- Impacto do Projeto Prático: verifica se o projeto prático auxiliou os alunos a entenderem melhor os padrões de projeto;
- Aplicação dos Princípios SOLID: avalia se a prática dos princípios SOLID contribuiu para a criação de código mais reutilizável e flexível;
- Confiança na Aplicação: mede a confiança dos alunos em aplicar padrões de projeto após a experiência educativa.

Todo o material desenvolvido para essa experiência foi revisado por um professor e um pesquisador em Engenharia de Software. Os materiais mencionados estão disponíveis para uso na seção de Disponibilidade de Artefatos

3.2 Análise dos Dados

Nesse relato de experiência, utilizaram-se análises quantitativas e qualitativas das respostas. Essa abordagem permitiu combinar perspectivas quantitativas e qualitativas para uma compreensão mais abrangente do impacto da prática de ensino. Para análise qualitativa, adotou-se a análise de conteúdo, de caráter exploratório, com categorização indutiva e validação por revisão entre pesquisadores, adequada ao escopo de um relato de experiência educacional.

Foi realizada uma análise estatística exploratória por meio do teste do Qui-Quadrado de independência, com o objetivo de identificar possíveis associações entre variáveis categóricas, como a experiência prévia dos alunos e a clareza das explicações teóricas. Esta análise estatística foi realizada por meio de um programa desenvolvido em *Python*, disponibilizado na seção Disponibilidade de Artefatos. Os dados qualitativos foram analisados a partir das respostas às questões abertas. A análise qualitativa foi inicialmente realizada pelo primeiro autor e revisada por mais dois autores deste trabalho.

4 Resultados

Essa seção apresenta os resultados sobre o ensino de padrões de projeto. Inicialmente, são apresentados o perfil dos participantes, sua familiaridade prévia com os conceitos abordados e, por fim, a avaliação da prática de ensino.

4.1 Perfil dos Participantes

Na primeira rodada, a turma possuía 36 alunos, enquanto na segunda rodada, a turma foi composta por 28 alunos. O formulário da primeira rodada foi respondido por 23 alunos. Na segunda rodada, o formulário de caracterização foi respondido por todos os alunos, enquanto o formulário de avaliação da disciplina foi respondido por 17 alunos. Cabe ressaltar que a participação nas atividades era obrigatória para a disciplina, no entanto, o preenchimento do formulário de avaliação era opcional. Nas duas rodadas, foi incluído um Termo de Consentimento Livre e Esclarecido (TCLE), por meio do qual os participantes eram informados sobre os objetivos da pesquisa e sobre o uso dos dados coletados. Com o objetivo de evitar viés, os formulários de avaliação foram aplicados após a liberação das notas pelo professor da disciplina.

Com base nos dados coletados na caracterização, as respostas foram consolidadas em dois gráficos, apresentados na Figura 2, que apresenta os dados coletados da primeira rodada, e na Figura 3, que apresenta os dados coletados da segunda rodada, com o objetivo de facilitar a análise dos diferentes aspectos avaliados dos perfis

dos alunos nas duas aplicações. Para padronizar a visualização, as variáveis, de ambas as figuras, foram organizadas em uma escala de A a D, em que A representa o nível mais baixo de familiaridade ou experiência, e D indica o nível mais alto.

Com os dados apresentados nas Figuras 2 e 3, pode-se notar que a maioria dos participantes, em ambas as turmas, não possui experiência profissional em desenvolvimento de software. A familiaridade com a programação orientada a objetos variou entre os participantes, predominando o nível básico de conhecimento, especialmente na segunda rodada. Além disso, os resultados indicam que a maioria dos alunos não possuía experiência prévia com padrões de projeto, sendo esse conhecimento restrito a poucos participantes. E que a maioria dos alunos não possuía conhecimento prévio sobre os princípios SOLID, o que evidencia a necessidade de introduzir esses conceitos na disciplina.

As linguagens de programação que se destacaram entre os participantes da segunda rodada foram *Python*, *Java* e *JavaScript*, como apresentado na Figura 4. Vale ressaltar que "Outros" engloba linguagens que apenas um participante marcou, entre elas SQL, HTML e CSS. Quanto às ferramentas utilizadas, destacam-se *Visual Studio Code*, *PyCharm*, *Git* e *IntelliJ IDEA*, apresentadas na Figura 5.

4.2 Avaliação da Metodologia Aplicada

Os participantes foram questionados sobre a eficácia da prática de ensino de padrões de projeto e princípios SOLID, bem como sobre a clareza das explicações teóricas e a complexidade das atividades propostas. As perguntas a seguir foram inspiradas no estudo de [7], que avaliou metodologias de ensino para padrões de projeto utilizando o modelo 4C/ID. No entanto, as questões foram adaptadas para o contexto específico desse trabalho, considerando as atividades realizadas durante o processo de aprendizagem.

Para fornecer uma visão consolidada das respostas dos participantes, os dados foram organizados em dois gráficos (Figuras 6 e 7). As respostas foram estruturadas em uma escala categórica de A a D, em que A representa a avaliação menos favorável e D a mais favorável, de acordo com o contexto de cada questão.

A prática de ensino foi avaliada positivamente pelos participantes em ambas as turmas, com resultados acima de 70%. E as explicações teóricas e os exemplos foram, em sua maioria, considerados suficientes ou moderadamente suficientes, indicando uma boa compreensão dos conteúdos apresentados.

Percebe-se também uma melhora na avaliação dos alunos em relação à complexidade das atividades propostas e ao projeto final da primeira para a segunda rodada, o que sugere que a redução da carga cognitiva dos alunos, por meio da retirada da obrigatoriedade da aplicação dos princípios SOLID nas atividades práticas, pode ter favorecido o aprendizado.

Os resultados indicam que o projeto prático contribuiu significativamente para o entendimento dos padrões de projeto, sendo amplamente avaliado de forma positiva pelos participantes. Os participantes perceberam uma contribuição relevante dos princípios SOLID e dos padrões de projeto na criação de código mais reutilizável e flexível. Essa percepção foi avaliada especificamente pelas perguntas 9 e 6 do primeiro questionário e do segundo questionário, respectivamente, relacionadas aos projetos práticos desenvolvidos pelos alunos.

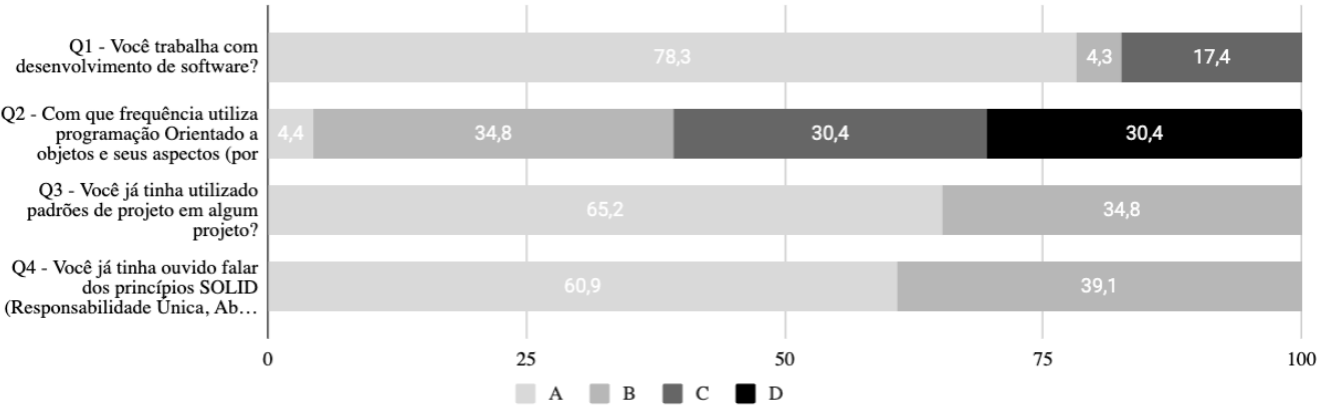


Figura 2: Caracterização dos participantes na primeira rodada

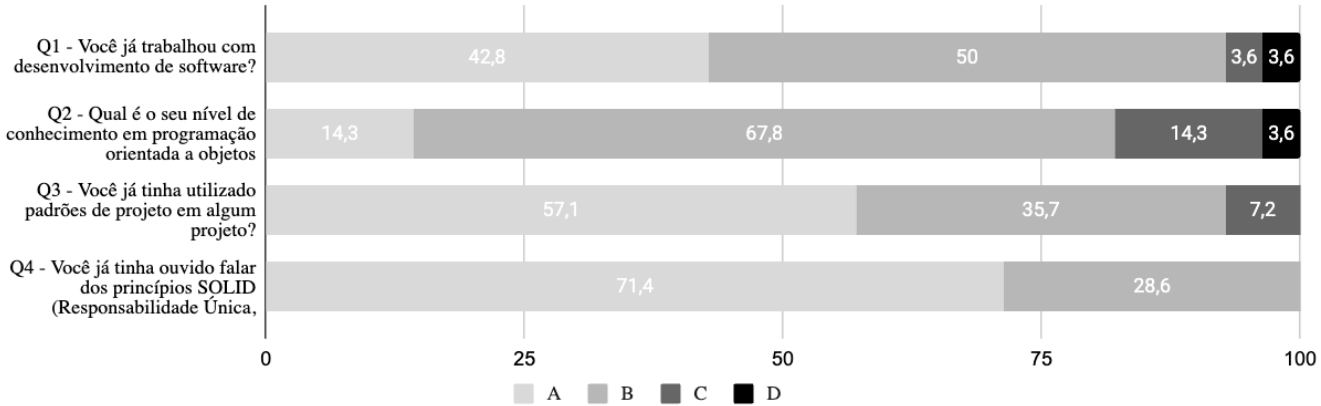


Figura 3: Caracterização dos participantes na segunda rodada

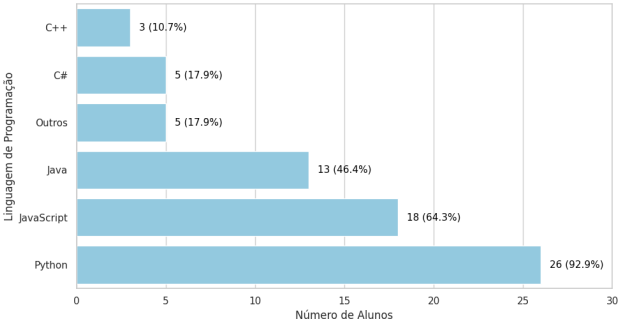


Figura 4: Linguagens de programação utilizadas pelos participantes da segunda rodada.

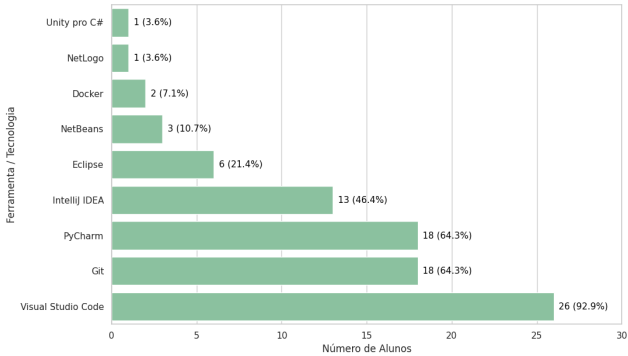


Figura 5: Ferramentas de desenvolvimento utilizadas pelos participantes da segunda rodada.

4.2.1 *Análise Estatística da Relação entre Variáveis.* Para verificar a existência de relações estatisticamente significativas entre as respostas dos participantes, foi aplicado o teste de Qui-Quadrado de

independência. Este teste permite identificar associações entre variáveis categóricas e avaliar se há evidências estatísticas que sustentem a interdependência entre elas. Para cada teste realizado, foram consideradas as seguintes hipóteses:

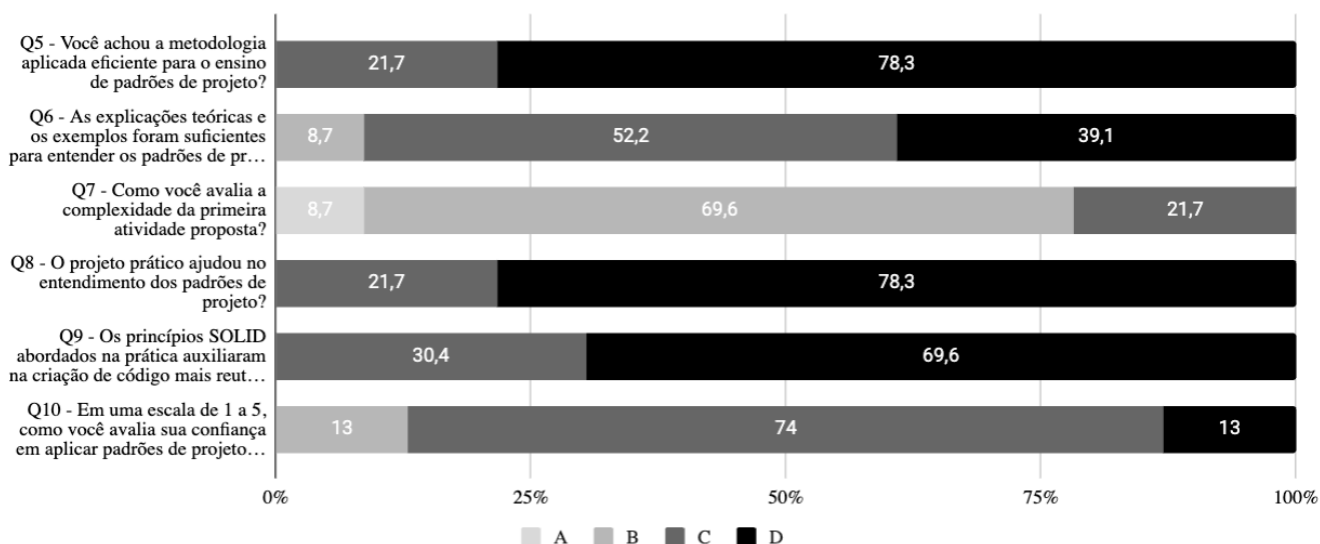


Figura 6: Avaliação dos participantes na primeira rodada

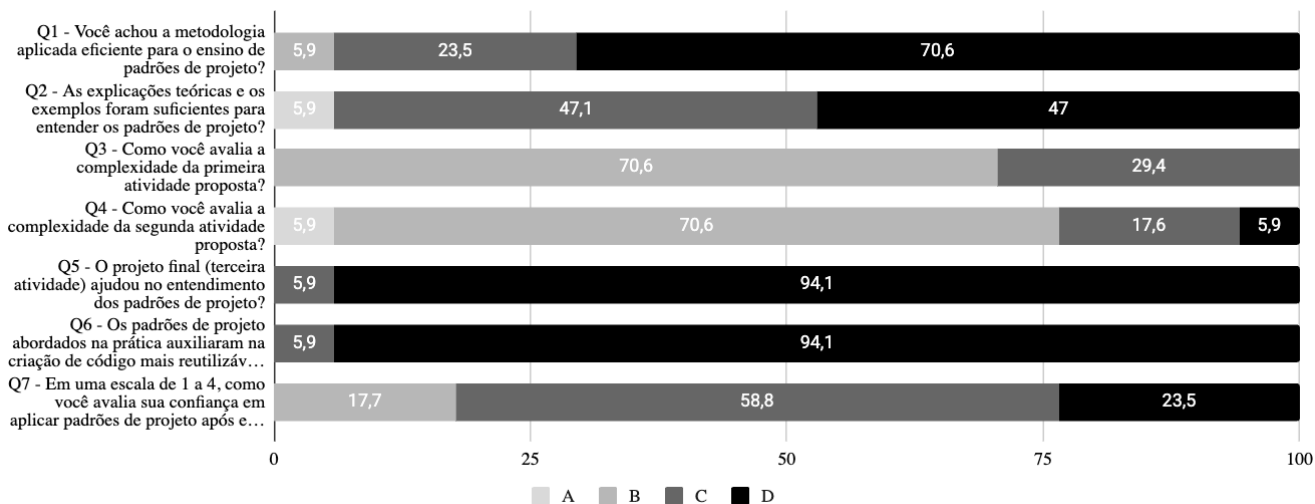


Figura 7: Avaliação dos participantes na segunda rodada

- H_0 (**hipótese nula**): não existe associação entre as variáveis analisadas, ou seja, elas são independentes.
- H_1 (**hipótese alternativa**): existe associação entre as variáveis analisadas, ou seja, elas são dependentes.

Adotou-se um nível de significância de 5% ($\alpha = 0,05$). Assim, para valores de p inferiores a esse limiar, a hipótese nula foi rejeitada, indicando evidência estatisticamente significativa de associação entre as variáveis [26]. Vale ressaltar que a interpretação dos valores de p deve ser realizada com cautela e em conjunto com o contexto da análise, conforme recomendado pela literatura.

Foi realizada uma análise da associação entre experiência em Desenvolvimento de Software e Percepções sobre o Processo de Aprendizagem. O objetivo desta análise foi investigar se a experiência prévia dos participantes com desenvolvimento de software influencia sua percepção sobre a clareza das explicações teóricas e o processo de aprendizagem dos padrões de projeto. Para isso, foram conduzidos testes de independência Qui-Quadrado, considerando todas as combinações entre as variáveis categóricas coletadas nas duas turmas. Em ambos os conjuntos de dados, foram testadas todas as possíveis associações entre as variáveis do questionário. Entretanto, neste trabalho são apresentados apenas os resultados

Tabela 1: Descrição das perguntas utilizadas na primeira rodada

Rodada	Pergunta	Descrição
1 ^a	Pergunta 1	As explicações teóricas e os exemplos foram suficientes para entender os padrões de projeto?
1 ^a	Pergunta 2	Você já tinha ouvido falar dos princípios SOLID (Responsabilidade Única, Aberto-Fechado, etc.)?
1 ^a	Pergunta 3	Você trabalha com desenvolvimento de software?
2 ^a	Pergunta 1	Como você avalia a complexidade da segunda atividade proposta?
2 ^a	Pergunta 2	Você já tinha utilizado padrões de projeto em algum projeto?
2 ^a	Pergunta 3	Você já trabalhou com desenvolvimento de software?

Tabela 2: Resultados significativos do teste Qui-Quadrado ($p < 0.05$) da primeira rodada

Variável 1	Variável 2	Qui-Quadrado	Valor-p	gl
Pergunta 2	Pergunta 1	7.8391	0.0199	2
Pergunta 2	Pergunta 3	11.2160	0.0242	4

Tabela 3: Resultados significativos do teste Qui-Quadrado ($p < 0.05$) da segunda rodada

Variável 1	Variável 2	Qui-Quadrado	Valor-p	gl
Pergunta 3	Pergunta 2	20.8125	0.0020	6
Pergunta 1	Pergunta 2	20.0303	0.0027	6

estatisticamente significativos, isto é, aqueles com valor-p inferior a 0,05.

Na primeira rodada, foram observadas associações significativas entre a experiência com desenvolvimento de software e as demais variáveis analisadas, incluindo percepção sobre a suficiência das explicações teóricas e familiaridade com conceitos de engenharia de software. A descrição das variáveis apresentadas nesta análise é apresentada na Tabela 1 e os resultados significativos são apresentados na Tabela 2.

Na segunda rodada, o mesmo procedimento foi adotado, com a realização de testes de independência entre todas as combinações possíveis das variáveis coletadas. Assim como na primeira turma, apenas os resultados estatisticamente significativos são reportados. Entretanto, observou-se que alguns alunos responderam apenas o formulário de caracterização, enquanto outros responderam exclusivamente o formulário de avaliação. Com o intuito de garantir a consistência da análise, foi realizada a integração das bases por meio do campo matrícula, permitindo a seleção apenas dos participantes que responderam a ambos os questionários. Dessa forma, os testes foram conduzidos exclusivamente sobre o subconjunto pareado da amostra. Os resultados são apresentados na Tabela 3. A descrição das variáveis apresentadas nesta análise é apresentada na Tabela 1.

Com um valor-p inferior a 0,05, é possível identificar uma associação estatisticamente significativa entre as perguntas avaliadas. Foi utilizado o teste Qui-Quadrado de independência por se tratar de variáveis categóricas, como níveis de familiaridade ou experiência prévia. O objetivo foi compreender se a atuação profissional influenciou a assimilação dos conteúdos.

5 Discussão

Os resultados foram analisados e interpretados com base na literatura sobre o ensino de padrões de projeto e princípios SOLID utilizando o modelo 4C/ID. A discussão aborda o impacto da metodologia utilizada, as dificuldades relatadas pelos alunos participantes e as sugestões propostas para aprimorar a prática de ensino.

5.1 Impactos da Prática de Ensino

O uso contínuo dos padrões em diversas atividades práticas foi evidenciado pelos alunos como fator crucial de aprendizagem, com sugestão de mais atividades ao longo da disciplina. Além disso, o trabalho em grupo e as apresentações finais favoreceram a troca de conhecimento, alinhado a Vahldick et al. [24].

A literatura também sustenta a eficácia de abordagens estruturadas, Silva et al. [20] observaram que o uso do padrão *Strategy*, sem suporte instrucional, pode prejudicar a manutenibilidade, Köppe [13] propôs o uso de padrões instrucionais para guiar o ensino, Stuurman e Florijn [23] defenderam a diversidade de exemplos concretos, Khomh e Gueheneuc [12] destacam o papel da experiência prévia na aplicação adequada dos padrões. Esses achados reforçam a adequação do 4C/ID para promover um ensino mais eficaz e contextualizado.

5.1.1 Vantagens da Abordagem Baseada em Prática. A metodologia aplicada, baseada no modelo 4C/ID, apresentou vantagens significativas para o ensino de padrões de projeto e princípios SOLID. Sua estrutura progressiva, que divide o aprendizado em tarefas de complexidade crescente, foi percebida pelos participantes como fator que facilitou a assimilação dos conceitos, reduzindo a carga cognitiva dos alunos [24].

A necessidade de aplicar os padrões em diferentes atividades e contextos parece ter contribuído para uma melhor compreensão dos benefícios práticos dessas técnicas, tornando o aprendizado mais concreto e alinhado com a realidade do desenvolvimento de software. Além disso, o uso de atividades incrementais reforçou a fixação do conhecimento, como destacado em Gómez-Martín et al. [11].

Outro benefício foi a organização do ensino em fases estruturadas, característica central do modelo 4C/ID. A divisão entre informação de suporte e informação procedural indica ter contribuído para os alunos desenvolverem autonomia na aplicação dos padrões, enquanto a prática de parte-tarefa pode ter favorecido a repetição necessária para consolidar os conceitos antes da implementação completa nos projetos finais [25]. Por fim, os resultados sugerem um aumento na confiança dos alunos para utilizar padrões de projeto em seus próprios códigos, alinhando-se ao que foi observado em [24], que destaca que metodologias progressivas contribuem para um aprendizado mais efetivo e duradouro.

A diferença entre 4C/ID e outras abordagens como *Problem-Based Learning* (PBL) ou *Project-Based Learning* (PBL) está no grau

de estrutura e suporte oferecido. Enquanto o PBL é orientado por problemas abertos e o PjBL baseia-se na autonomia dos alunos ao longo de um projeto, o 4C/ID organiza a aprendizagem em tarefas autênticas com níveis progressivos de dificuldade, articuladas com apoio teórico, instruções detalhadas e prática focada. Essa abordagem demonstrou-se potencialmente mais adequada para alunos com pouca vivência prática, por oferecer uma rota de aprendizado mais estruturada e guiada.

5.2 Dificuldades Enfrentadas Pelos Alunos

Embora a metodologia tenha sido bem recebida, alguns participantes relataram dificuldades, especialmente na primeira rodada da aplicação. Os testes de Qui-Quadrado evidenciaram associações entre a experiência prévia dos alunos no mercado de trabalho e a complexidade das atividades propostas, bem como entre a familiaridade prévia com padrões de projeto e sua percepção sobre a suficiência das explicações teóricas.

Muitos tiveram problemas para entender e aplicar alguns princípios SOLID, principalmente os que envolvem modularidade e separação de responsabilidades. Devido a essas dificuldades, decidiu-se retirar a obrigatoriedade do uso dos princípios SOLID, a fim de dar maior ênfase no aprendizado sobre padrões de projeto. Isso corresponde ao exposto por Silva [14], que destaca que a abstração dos princípios SOLID pode ser desafiadora para iniciantes e exige uma base sólida em design de software.

Outro ponto levantado foi o tempo para aprender e aplicar os conceitos de forma estruturada. Alguns alunos sugeriram o aumento da quantidade de atividades práticas ao longo do curso, com o objetivo de reforçar os padrões de maneira mais gradual, o que está alinhado com Vahldick et al. [24], que propõem o modelo 4C/ID como uma abordagem eficaz para fragmentar o ensino de padrões de projeto e otimizar a assimilação do conteúdo.

5.3 Sugestões dos Participantes e Possíveis Melhorias

Os alunos sugeriram várias melhorias para o ensino de padrões de projeto, principalmente a inclusão de mais exemplos práticos, mais tempo para a realização dos projetos e um acompanhamento mais próximo por meio de *checkpoints*. Essas sugestões estão alinhadas com Vahldick et al. [24], que destacam a importância de um ensino estruturado em etapas, permitindo que os alunos assimilem os conceitos gradualmente e os apliquem de forma mais eficiente. O modelo 4C/ID enfatiza a necessidade de dividir o aprendizado em componentes menores, para um desenvolvimento progressivo das habilidades relacionadas à aplicação dos padrões de projeto.

Além disso, alguns participantes destacaram a importância de uma abordagem mais orientada à prática, sugerindo atividades que incentivem a experimentação e a aplicação direta dos padrões de projeto. Estudos como os de Silveira [21] indicam que métodos baseados em resolução de problemas e estudo de casos reais contribuem para uma melhor assimilação dos conceitos. Quando combinadas com a estruturação do ensino baseada no modelo 4C/ID, possibilitam um aprendizado mais progressivo e eficaz, permitindo que os alunos desenvolvam suas habilidades de forma mais natural e aplicada ao contexto do desenvolvimento de software.

5.4 Considerações Sobre a Efetividade da Metodologia

A efetividade da prática de ensino não foi medida por um indicador agregado, mas pela análise da percepção dos alunos e pela qualidade das entregas avaliadas pelos autores para atribuição das notas finais. A combinação entre teoria e prática foi fundamental para a compreensão dos conceitos, reforçando a importância do aprendizado experiencial e alinhando-se à proposta do 4C/ID de etapas progressivas. Por outro lado, os resultados também apontam a necessidade de ajustes na organização e no tempo do curso, especialmente na forma como o conteúdo é apresentado e no suporte oferecido durante a implementação dos padrões. A introdução de mais atividades e um cronograma mais flexível podem tornar o aprendizado mais acessível e equilibrado.

5.5 Implicações para o Ensino de Padrões de Projeto

Os resultados trazem *insights* para tornar o ensino de princípios e padrões de projeto mais eficiente e acessível. A prática continua se mostrou essencial para a assimilação dos conceitos, reforçando a importância de metodologias que incentivem a experimentação e a aplicação em cenários reais. Isso está alinhado com o modelo 4C/ID, que sugere dividir o aprendizado em etapas menores e progressivas, facilitando o desenvolvimento das habilidades necessárias para a aplicação dos padrões.

Com base nas sugestões dos participantes e na literatura existente, destacam-se quatro melhorias possíveis: mais exercícios práticos curtos antes de projetos complexos, exemplos de código reais, checkpoints mais frequentes para feedback contínuo, e um ritmo de aprendizado mais ajustável ao longo das aulas.

Dessa forma, o ensino de princípios e padrões de projeto pode se tornar mais acessível e eficiente, preparando melhor os alunos para desafios práticos no desenvolvimento de software e sua qualidade. Os materiais utilizados no estudo, como projetos, slides e formulário de avaliação, estão disponíveis publicamente na seção “Disponibilidade de Artefatos” deste artigo. Para adaptação a outros contextos educacionais, é necessário considerar fatores como o perfil dos alunos, o tempo disponível na disciplina e o nível de maturidade técnica [24]. Por exemplo, turmas mais experientes podem receber projetos com maior complexidade, enquanto turmas iniciantes podem exigir acompanhamento mais intensivo e menor densidade de conteúdo. Essa flexibilidade torna a proposta aplicável em diferentes cursos e níveis de ensino técnico e superior [6].

5.6 Limitações do Trabalho

Esse trabalho apresenta algumas limitações que devem ser consideradas. Uma delas diz respeito à diversidade de níveis de experiência dos participantes, o que pode ter influenciado a forma como absorveram e aplicaram os padrões de projeto. Alunos com maior familiaridade com desenvolvimento de software, especialmente com Programação Orientada a Objetos (POO), tenderam a apresentar maior facilidade na implementação, enquanto aqueles com menos experiência nessa abordagem enfrentaram desafios adicionais. Os resultados do teste de Qui-Quadrado indicam a existência de associações estatisticamente significativas entre a experiência prévia dos

participantes e suas percepções sobre o processo de aprendizagem, incluindo a avaliação da clareza das explicações teóricas.

Outra limitação está na complexidade dos padrões abordados, que exigem um tempo maior para assimilação e prática. A adoção de uma abordagem progressiva, como propõe o modelo 4C/ID, pode ajudar a estruturar melhor o aprendizado, permitindo que os alunos avancem de forma mais gradual. Em relação à execução da experiência, destaca-se o curto período de avaliação, que impossibilita medir os efeitos da metodologia no longo prazo. Por fim, destaca-se que esta é a experiência conduzida em um número limitado de turmas e que os resultados apresentados não podem ser generalizados. Porém, podem contribuir para uma reflexão sobre o ensino de princípios e padrões de projeto.

Outra limitação refere-se ao possível uso de ferramentas de geração de código por parte dos alunos durante a realização das atividades. Embora a apresentação do projeto final tenha sido utilizada para avaliar a capacidade dos alunos de compreender e explicar as soluções implementadas, não é possível garantir que todo o código produzido tenha sido desenvolvido sem o auxílio de ferramentas.

Por fim, vale destacar que os participantes eram alunos da disciplina ministrada por um dos autores, o que pode ter influenciado positivamente as respostas ao questionário, mesmo sendo opcional e aplicado após a divulgação das notas.

6 Conclusões e Trabalhos Futuros

Esse trabalho teve como objetivo avaliar a eficácia do uso do modelo 4C/ID no ensino de padrões de projeto e princípios SOLID. Como principais contribuições, destacam-se: (i) a adoção e avaliação de uma estrutura didática incremental baseada no 4C/ID; (ii) o uso da refatoração de projetos legados e da prática repetida como estratégia prática de ensino, promovendo aprendizagem contextualizada e próxima da realidade do mercado; (iii) a evolução da metodologia a partir de múltiplas aplicações, incorporando ajustes baseados no *feedback* dos alunos; e (iv) a disponibilização pública dos artefatos educacionais desenvolvidos, visando à reutilização por outros docentes [25].

Os resultados demonstraram que uma maior ênfase na prática foi um fator determinante para o entendimento dos padrões de projeto, permitindo que os alunos compreendessem melhor sua aplicação no desenvolvimento de software. A necessidade de implementar os padrões diretamente no código e justificá-los durante o desenvolvimento do projeto proporcionou uma experiência mais concreta e alinhada às necessidades do mercado. Além disso, a interação em equipe e as apresentações dos projetos contribuíram para a troca de conhecimento e o reforço do aprendizado, o que está de acordo com os achados na literatura, que destacam a importância do aprendizado ativo e da colaboração para a fixação dos conceitos. A análise dos dados reforça a eficácia do modelo 4C/ID para o ensino de padrões de projeto, pois sua abordagem estruturada permite que os alunos desenvolvam habilidades de forma progressiva, consolidando conceitos fundamentais antes de avançar para aplicações mais complexas.

Como principais lições aprendidas, destacamos três pontos: (i) o modelo 4C/ID mostrou-se especialmente eficaz para turmas com baixa experiência prática, fornecendo uma estrutura clara, guiada

e progressiva; (ii) a prática incremental e o suporte contínuo facilitaram a transição dos alunos da teoria para a aplicação concreta de padrões de projeto; e (iii) a necessidade de aplicar os padrões em refatorações reais, mesmo em projetos simples, foi percebida como fundamental para consolidar o aprendizado. Esses achados são especialmente relevantes para instituições que enfrentam o desafio de ensinar conceitos avançados de design a estudantes com pouca vivência em desenvolvimento de software.

A experiência da primeira rodada indicou que a tentativa de paralelizar o ensino de padrões de projeto e dos princípios SOLID pode aumentar a carga cognitiva dos alunos. Por esse motivo, optou-se por abordar apenas os padrões de projeto nas atividades práticas da segunda rodada. Essa mudança pode ter influenciado o feedback dos alunos em relação ao impacto do projeto prático entre as duas rodadas, uma vez que eles puderam concentrar seus esforços na aplicação de apenas um dos dois conceitos.

Para estudos futuros, sugere-se a ampliação do tempo dedicado à prática dos princípios e padrões de projeto ao longo do curso, permitindo que os alunos tenham mais oportunidades de experimentar e consolidar os conceitos estudados. Além disso, a adoção de um maior número de atividades, seguindo os princípios do modelo 4C/ID, pode facilitar a assimilação dos padrões, garantindo que a base em Programação Orientada a Objetos seja fortalecida antes da introdução de conceitos mais avançados.

Outra recomendação é o aprimoramento das estratégias de acompanhamento e suporte aos alunos, garantindo que dificuldades sejam identificadas e corrigidas ao longo do processo. A divisão das explicações teóricas em um maior número de aulas, a adoção de *checkpoints* mais frequentes e a oferta de momentos específicos para revisões e ajustes no código podem tornar o aprendizado mais eficiente e acessível para todos os níveis de experiência. Também para estudos futuros, é recomendável investigar a aplicação de práticas estruturadas em outras abordagens de ensino. Isso pode incluir, por exemplo, a utilização de jogos ou ferramentas de aprendizagem, a fim de determinar como essas diferentes estratégias podem complementar o modelo 4C/ID e potencializar o ensino de padrões de projeto e princípios SOLID. Com isso, espera-se que este estudo contribua para a disseminação do ensino de padrões de projeto, fornecendo subsídios para metodologias adequadas na formação de profissionais mais preparados para os desafios do mercado.

Disponibilidade de Artefatos

- Todos os materiais utilizados nesse relato de experiência estão disponíveis em: <https://doi.org/10.5281/zenodo.21417864>;

Agradecimentos

O presente trabalho foi realizado com apoio da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Código de Financiamento 001. Adicionalmente, o último autor agradece ao CNPq (Processos 311533/2025-6 e 448761/2025-4).

Referências

- [1] Zain Azimullah, Young Sun An, and Paul Denny. 2020. Evaluating an Interactive Tool for Teaching Design Patterns. In *Proceedings of the Twenty-Second Australasian Computing Education Conference*. ACM, 167–176. doi:10.1145/3373165.3373184

- [2] B. Bafandeh Mayvan, A. Rasoolzadegan, and Z. Ghavidel Yazdi. 2017. The state of the art on design patterns: A systematic mapping of the literature. *J. Syst. Softw.* 125, C (March 2017), 93–118. doi:10.1016/j.jss.2016.11.030
- [3] Raphael Cabral and Marcos Kalinowski. 2024. Investigating the Impact of SOLID Design Principles on Machine Learning Code Understanding. In *Anais do XXIII Simpósio Brasileiro de Qualidade de Software* (Bahia/BA). SBC, Porto Alegre, RS, Brasil, 703–705. <https://sol.sbc.org.br/index.php/sbqs/article/view/33000>
- [4] Caroline Teixeira da Cruz, Mario Augusto Pazoti, and Francisco Virginio Maracci. 2016. LEPATTERN - UMA FERRAMENTA PARA ENSINO DE PADRÕES DE PROJETO: IMPLEMENTAÇÃO DOS PADRÕES STRATEGY E FACADE. *Colloquium Exactarum*. ISSN: 2178-8332 8, 1 (jun. 2016), 35–47. <https://journal.unoest.e.br/index.php/ce/article/view/1550>
- [5] Wilian DÁvila and Lucia Giraffa. 2023. Ensino de Programação Orientada a Objetos Para Iniciantes: Uma Metodologia para Programação Criativa. In *Anais do XXXIV Simpósio Brasileiro de Informática na Educação* (Passo Fundo/RS). SBC, Porto Alegre, RS, Brasil, 335–344. doi:10.5753/sbie.2023.235067
- [6] Rodrigo Silva e Elisangela Miranda e Thiago Pereira. 2016. Uma Experiência no Ensino de Padrões de Projeto Utilizando Jogo no Processo de Ensino-Aprendizagem. *Anais dos Workshops do Congresso Brasileiro de Informática na Educação* 5, 1 (2016), 1218. <http://milanesa.ime.usp.br/rbie/index.php/wcbie/article/view/7047>
- [7] Adilson Vahldick e Pablo Schoeffel e Paolo Moser. 2019. Metodologia de Ensino de Padrões de Projeto Baseado no Modelo 4C/ID. *Anais dos Workshops do Congresso Brasileiro de Informática na Educação* 8, 1 (2019), 397. <http://milanesa.ime.usp.br/rbie/index.php/wcbie/article/view/8981>
- [8] Elisabeth Freeman, Eric Freeman, Bert Bates, and Kathy Sierra. 2004. *Head First Design Patterns*. O'Reilly & Associates, Inc. <https://dl.acm.org/doi/10.5555/1076324>
- [9] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. 2000. *Padrões de Projeto: Soluções reutilizáveis de software orientado a objetos*. Vol. 11. Bookman.
- [10] Lina Garcés and Edson Oliveira. 2025. Teaching Software Design Patterns: An Experience Using the Jigsaw Classroom. In *Proceedings of the XXXIX Brazilian Symposium on Software Engineering (SBES 2025)* (Recife, PE, Brazil). ACM.
- [11] Marco Antonio Gómez-Martin, Guillermo Jiménez-Díaz, and Javier Arroyo. 2009. Teaching design patterns using a family of games. *Association for Computing Machinery* (2009), 268–272. doi:10.1145/1562877.1562960
- [12] Foutse Khomh and Yann-Gael Gueheneuc. 2008. Do Design Patterns Impact Software Quality Positively?. In *2008 12th European Conference on Software Maintenance and Reengineering*. 274–278. doi:10.1109/CSMR.2008.4493325
- [13] Christian Köppe. 2011. A pattern language for teaching design patterns (part 1). *Association for Computing Machinery*, Article 2 (2011), 21 pages. doi:10.1145/2396716.2396718
- [14] Robert C. Martin. 2000. Design Principles and Design Patterns. *Object Mentor* 1, 34 (2000), 597.
- [15] Robert C. Martin. 2003. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall PTR, USA. <https://dl.acm.org/doi/10.5555/515230>
- [16] Robert C Martin. 2009. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- [17] Robert C. Martin. 2017. *Clean Architecture: A Craftsman's Guide to Software Structure and Design* (1st ed.). Prentice Hall Press, USA. <https://dl.acm.org/doi/book/10.5555/3175742>
- [18] André Menolli and Bruno Strik. 2025. Educational Insights from Code: Mapping Learning Challenges in Object-Oriented Programming through Code-Based Evidence. In *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software* (Recife/PE). SBC, Porto Alegre, RS, Brasil, 544–554. doi:10.5753/sbes.2025.11095
- [19] Lucas Reis, Fernando Castro, Hudson Diniz, Benedito Mendes, Luis Silva, and Davi Viana. 2024. Integrating DevOps into Software Engineering Education: A Laboratory Experience. In *Proceedings of the XXIII Brazilian Symposium on Software Quality (SBQS '24)*. Association for Computing Machinery, New York, NY, USA, 635–645. doi:10.1145/3701625.3701688
- [20] Gabriel Silva, Vinícius Andrade, Reginaldo Ré, and Rafael Meneses. 2021. A Quasi-Experiment to Investigating the Impact of the Strategy Design Pattern on Maintainability. In *Proceedings of the XXXV Brazilian Symposium on Software Engineering* (Joinville, Brazil) (SBES '21). Association for Computing Machinery, New York, NY, USA, 105–114. doi:10.1145/3474624.3474636
- [21] Ismar Silveira. 2016. A Game Development-based strategy for Teaching Software Design Patterns through Challenge-Based Learning under a Flipped Classroom approach. *SBC* (2016), 1996–2005. doi:10.5753/wei.2016.9643
- [22] Sylvia Stuurman and Gert Florijn. 2004. Experiences with teaching design patterns. In *Proceedings of the 9th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education* (Leeds, United Kingdom) (ITICSE '04). Association for Computing Machinery, New York, NY, USA, 151–155. doi:10.1145/1007996.1008037
- [23] Sylvia Stuurman and Gert Florijn. 2004. Experiences with teaching design patterns. *SIGCSE Bull.* 36, 3 (June 2004), 151–155. doi:10.1145/1026487.1008037
- [24] Adilson Vahldick, Schoeffel Pablo, and Moser Paolo. 2019. Metodologia de Ensino de Padrões de Projeto Baseado no Modelo 4C/ID. In *Anais do VIII Congresso Brasileiro de Informática na Educação (CBIE 2019) (Anais do Workshop de Computação na Educação (WCBIE))*. <http://milanesa.ime.usp.br/rbie/index.php/wcbie/article/view/8981>
- [25] Jeroen J. G. van Merriënboer. 2019. *The Four-Component Instructional Design Model: An Overview of its Main Design Principles*. Technical Report. School of Health Professions Education, Faculty of Health, Medicine and Life Sciences, Maastricht University, Maastricht, The Netherlands. Available at <https://www.4cid.org/wp-content/uploads/2021/04/vanmerrienboer-4cid-overview-of-main-design-principles-2021.pdf>.
- [26] Ronald L. Wasserstein and Nicole A. Lazar. 2016. The ASA Statement on p-Values: Context, Process, and Purpose. *The American Statistician* 70, 2 (2016), 129–133. doi:10.1080/00031305.2016.1154108 arXiv:<https://doi.org/10.1080/00031305.2016.1154108>